

Contents

1	Generalità	2
1.1	Gestione	2
1.2	Indirizzi di memoria	2
1.2.1	Binding degli indirizzi	2
1.2.2	MMU	3
1.2.3	Caricamento	4
1.2.4	Collegamento	4
1.2.5	Swapping	5
1.2.6	Considerazioni	6
2	Allocazione Contigua	6
2.1	Concetti Generali	6
2.2	Partizioni Fisse	7
2.2.1	Allocazione della Memoria	7
2.2.2	Supporto per Rilocazione	8
2.2.3	Valutazione	8
2.3	Partizioni variabili	9
2.3.1	Allocazione della Memoria	9
2.3.2	Supporto per Rilocazione	9
2.3.3	Valutazione	9
2.3.4	Compattazione	9
2.4	Buddy System	10
2.4.1	Valutazione	10
3	Paginazione	11
3.1	Generalità	11
3.2	Architettura	11
3.2.1	Gestione dei processi	12
3.3	Page Table	12
3.3.1	Page Table con TLB	13
3.3.2	Protezione	13
3.3.3	Struttura	14
3.3.4	Pagine Condivise	15
4	Segmentazione	16
4.1	Generalità	16
4.2	Architettura	17
4.3	Considerazioni	17
5	Segmentazione Paginata	18
5.1	Generalità	18
5.2	Architettura	18

1 Generalità

1.1 Gestione

Problematiche da affrontare:

- **allocazione** singoli job / processi in memoria
- **protezione** spazio di indirizzamento
- **condivisione** spazio di indirizzamento
- gestione **swapping**

Vincolo di funzionamento: per essere eseguito un programma deve essere portato in memoria e *trasformato* in un processo:

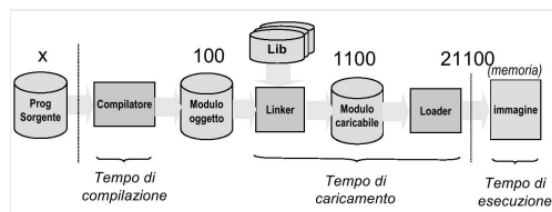
- CPU preleva istruzioni da eseguire da memoria in base a valore del P.C.
- ogni istruzione decodificata, può prevedere prelievo di operandi dalla memoria
- al termine di esecuzione dell'istruzione risultato può dover essere scritto in memoria
- al termine del processo la “sua” memoria viene rilasciata

1.2 Indirizzi di memoria

Trasformazione programma in processo implica assunzioni di diversa semantica di indirizzi e spazio di memoria → introduzione distinzione tra **spazio indirizzi logici** e **spazio indirizzi fisici**.

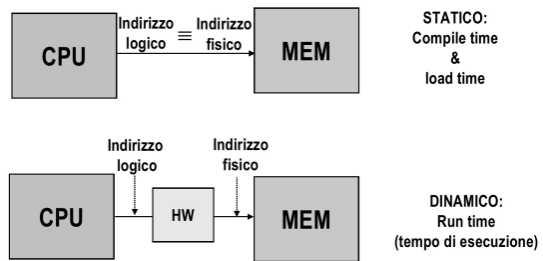
1.2.1 Binding degli indirizzi

- indirizzi del programma sorgente sono simbolici
- durante compilazione si associano a indirizzi simbolici indirizzi **rilocabili**
- in fase collegamento o caricamento si associano a indirizzi rilocabili indirizzi assoluti



Il binding di dati/istruzioni a indirizzi assoluti può avvenire in diversi momenti:

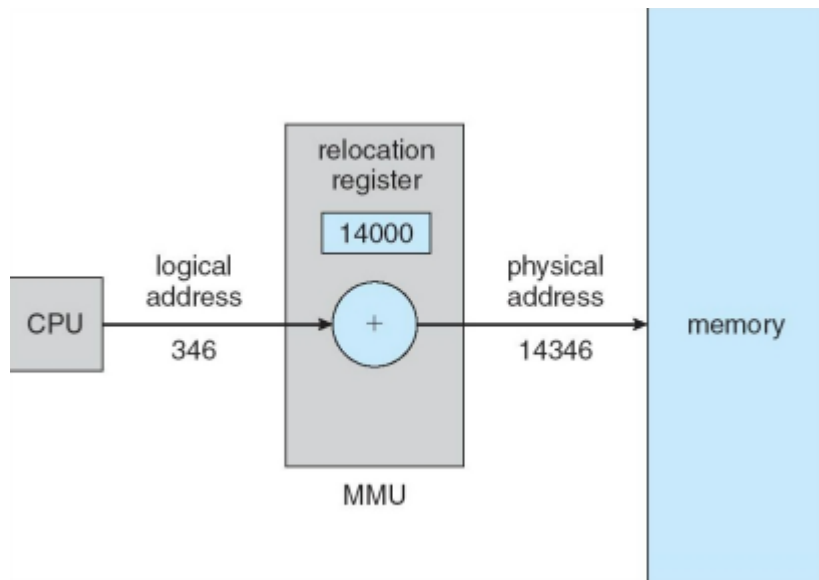
- Al tempo di compilazione (*binding statico al compile time*):
 - Se si conosce a priori la posizione in memoria, si può generare un **codice assoluto**.
 - Se si vuole cambiare posizione, è necessario **ricompilare**.
- Al tempo di caricamento (*binding statico al load time*):
 - Serve generare un **codice rilocabile** (repositionabile).
 - Gli indirizzi sono **relativi** all'inizio del programma.
 - Per cambiare posizione, basta **ricaricare**.
- Al tempo di esecuzione (*binding dinamico al run time*):
 - Il binding è **posticipato**, il processo può essere spostato anche durante l'esecuzione.
 - Richiede **supporto hardware** (es. registri base e limite).



Un HW particolare effettua il binding dinamico: la **MMU**

1.2.2 MMU

Contenuto di un particolare registro **registro di rilocazione** viene aggiunto a ogni indirizzo riferito da un processo, con l'indirizzo risultante si accede alla memoria:



1.2.3 Caricamento

- Statico: intero codice caricato in memoria per procedere all'esecuzione
- Dinamico: cerca di migliorare utilizzo della memoria centrale
 - moduli di codice caricati al momento del primo utilizzo
 - conseguenza: parti di codice non utilizzate non vengono caricate → "risparmio" memoria
 - tecnica utile con codici suddivisibili in parti che gestiscono casi specifici (error handling)

Solitamente caricamento dinamico richiede intervento minimo da parte del S.O. (può essere realizzato a livello di linguaggio/programma)

1.2.4 Collegamento

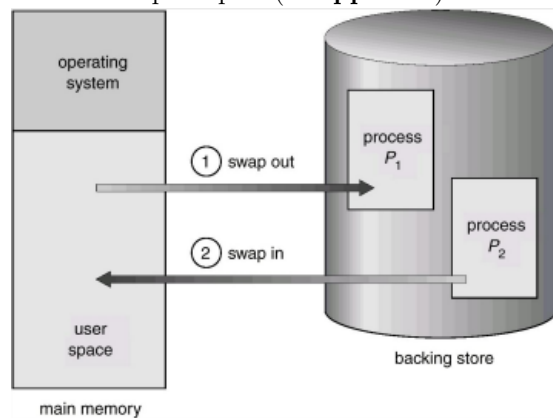
- Statico: tutti i riferimenti del codice definiti prima dell'avvio dell'esecuzione. Immagine del processo contiene interamente il codice di eventuali librerie
- Dinamico: utilizzato solamente per il codice delle librerie
 - linking posticipato al tempo di esecuzione
 - conseguenza: codice del programma non include codice delle librerie, ma solo dei riferimenti (*stub*)
 - *stub* indica dove trovare codice della libreria alla prima necessità (una volta caricata *stub* viene rimpiazzato con sua locazione in memoria)

- tutti i processi eseguono un'istanza dello stesso codice di librerie (singola copia in memoria)

Linking dinamico richiede intervento essenziale del S.O. (può interessare spazi di memoria di processi diversi).

1.2.5 Swapping

Tecnica che permette di rimuovere un processo da memoria principale e spostarlo in memoria secondaria (**swapped out**), in seguito questo potrà essere ricaricato in memoria principale (**swapped in**).



- tempo di swapping proporzionale a ammontare dei dati trasferiti
- sensato in caso di multiprogrammazione (altrimenti dovrebbe essere attivato ad ogni context switch)
- di norma è necessario supporto a binding dinamico per poter ri-locare i processi al momento dello swap-in
- con binding statico invece va ricaricato il processo sempre nella stessa posizione
- moderni S.O. contemplano lo swapping (in qualche forma)

La tecnica definita finora è detta **swapping standard**, mentre alcuni S.O. ne adottano forme diverse:

- per evitare di trasferire intero processo molti S.O. moderni effettuano swapping solo su porzioni del processo (*pagine*) → tecnica connessa a uso della *paginazione*
- S.O. di dispositivi mobili non operano swapping di processi, poiché:
 - memoria di solito è flash, assenza di HD

- memoria flash presenta un limite al numero di scritture, swapping degraderebbe la memoria
- se si necessita liberare memoria il S.O., invece di effettuare swapping, richiede a app di rinunciare a parte di memoria allocata (o sottrae a app pagine di memoria non allocate / termina la app)

1.2.6 Considerazioni

- in presenza di multiprogrammazione è impossibile fissare staticamente locazione ove processi vengono caricati
- presenza di swapping impone impiego di rilocalizzazione dinamica

Sono necessarie tecniche specifiche per gestione memoria principale, in caso il programma sia **interamente** caricato in memoria principale le tecniche "main" sono:

- Allocazione contigua
- Paginazione
- Segmentazione
- Segmentazione Paginata

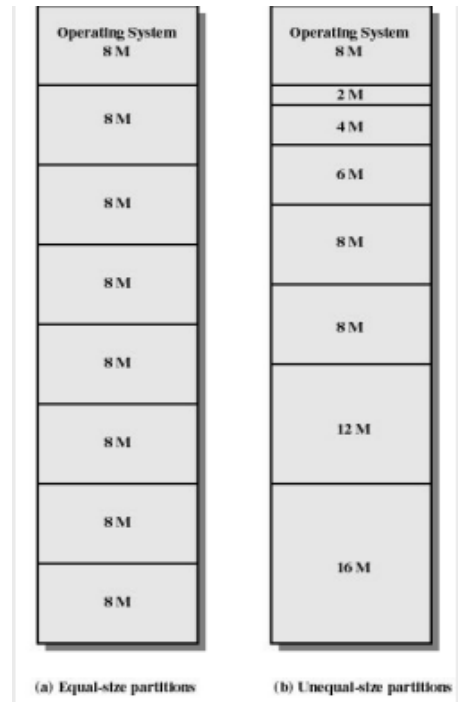
2 Allocazione Contigua

2.1 Concetti Generali

- processi allocati in memoria in posizioni **contigue** (ogni processo occupa un'**unica** sezione di memoria pari alla sua dimensione)
- la memoria viene suddivisa in due "zone":
 - una utilizzata dal S.O. (contenente vettore delle interruzioni)
 - una dedicata ai processi utente
- memoria quindi suddivisa in partizioni disgiunte (ognuna accoglierà un processo), due varianti:
 - Allocazione contigua con **partizioni fisse**
 - Allocazione contigua con **partizioni variabili**

2.2 Partizioni Fisse

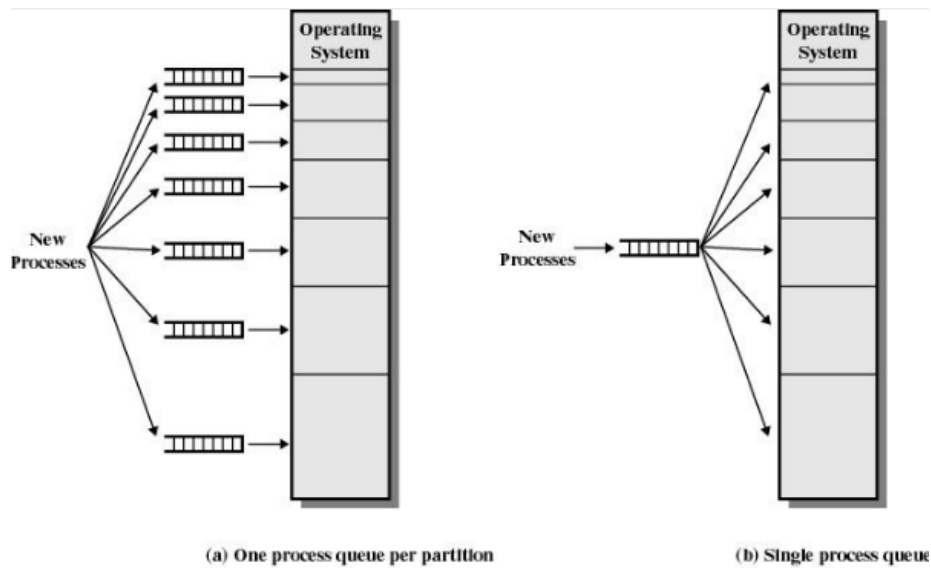
- Memoria suddivisa in un insieme di partizioni di dimensioni prefissate (non necessariamente uguali)
- Scelta di quante partizioni / dimensioni viene fatta una volta per tutte
- Un processo viene caricato in una partizione in grado di contenerlo interamente



2.2.1 Allocazione della Memoria

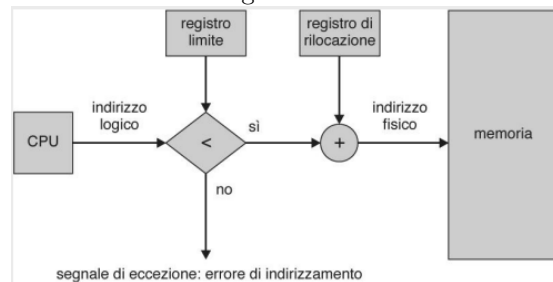
Long-term scheduling determina partizione in cui caricare un processo, esistono due tecniche base:

- una coda per ciascuna partizione:
 - alla creazione un processo viene assegnato a coda relativa alla partizione più piccola in grado di contenerlo
 - limitata flessibilità (possono esserci più processi in coda e partizioni non utilizzate)
- una coda singola: processo allocato nella partizione libera più piccola tra quelle in grado di contenerlo (in caso non esistano → swap-out)



2.2.2 Supporto per Rilocalione

MMU consiste di registri base e limite:



2.2.3 Valutazione

- Vantaggi
 - semplice da implementare
 - basso carico di gestione da parte di S.O.
- Svantaggi
 - grado di multiprogrammazione fissato
 - si genera **frammentazione**
 - * *interna* (se dimensione della partizione superiore a dimensione del processo che vi si carica)
 - * *esterna* (se vi sono partizioni non utilizzato perché non soddisfano esigenze dei processi in coda)

2.3 Partizioni variabili

Si vorrebbe partizionare memoria in parti di dimensioni corrispondenti a dimensioni dei processi che verranno caricati, ottenendo così eliminazione della frammentazione interna. Questo risultato si ottiene non fissando a priori numero e dimensioni delle partizioni.

2.3.1 Allocazione della Memoria

Si vede la memoria come un insieme di **buchi** (hole), ognuno di questi è una sezione di memoria. S.O. mantiene informazioni su partizioni assegnate e buchi disponibili. Un processo viene allocato in uno dei buchi in grado di contenerlo. Dato che possono esserci più buchi quando un processo deve essere caricato c'è il problema di scegliere in che buco allocarlo, 3 soluzioni:

- First-Fit (si occupa primo buco abbastanza grande che si trova)
- Best-Fit (si occupa buco più piccolo tra quelli in grado di contenere il processo)
- Worst-Fit (si occupa buco più grande)

2.3.2 Supporto per Rilocalizzazione

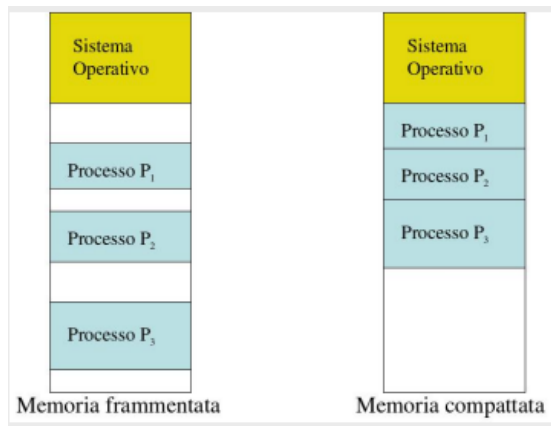
MMU con stessa struttura della tecnica con partizioni fisse

2.3.3 Valutazione

- Vantaggi
 - non esiste frammentazione interna
 - grado di multiprogrammazione non prefissato
- Svantaggi
 - frammentazione esterna può essere problematica

2.3.4 Compattazione

Soluzione a problema della frammentazione esterna: rilocalizzare processi e compattare spazio libero ottenendo unico buco (possibile solo con rilocalizzazione dinamica)



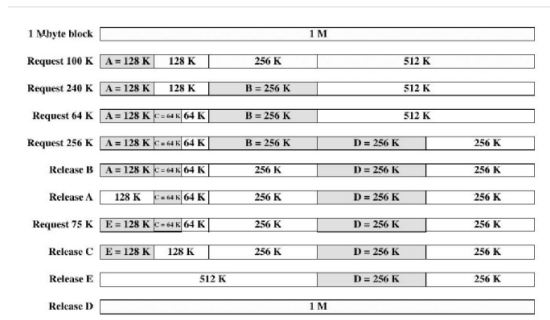
2.4 Buddy System

Compromesso tra partizioni fisse e variabili. Sia 2^m dimensione del più piccolo buco che si vuole allocare e 2^M tutta la memoria, allora:

- memoria vista come insieme di liste di blocchi di dimensione 2^k con $m \leq k \leq M$
- memoria disponibile è quindi insieme di blocchi di dimensione 2^i (per ogni i)
- all'inizio la lista dei blocchi di dimensione 2^M contiene un solo blocco (tutta la memoria) mentre altre liste sono vuote
- quando giunge richiesta di dimensione s , si crea blocco con dimensione sufficiente (potenza di 2):
 - se $2^{M-1} < s \leq 2^M$ si alloca intera memoria
 - altrimenti blocco di dimensione 2^M viene diviso in due blocchi di dimensione 2^{M-1} e si verifica se $2^{M-2} < s < 2^{M-1}$, ...
- quando un processo rilascia suo blocco questo viene inserito nell'opportuna lista di blocchi liberi
- quando due blocchi adiacenti di dimensione 2^i si liberano, possono essere fusi in un blocco di dimensione 2^{i+1}

2.4.1 Valutazione

- Vantaggi: compattazione si effettua scorrendo lista dei blocchi
- presenza di frammentazione interna (non è detto che processi abbiano sempre dimensione 2^i)



3 Paginazione

3.1 Generalità

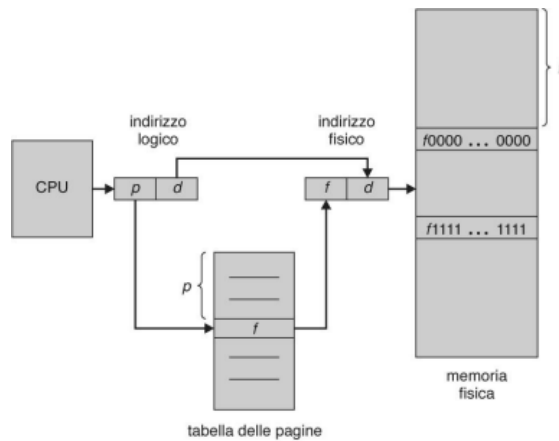
Introdotta per eliminare frammentazione esterna.

- Idea: permettere che un processo **non sia allocato in modo contiguo**
- memoria fisica organizzata in blocchi di dimensione fissa (**frame**)
- memoria logica organizzata in blocchi della stessa dimensione detti **pagine**
- S.O. tiene traccia dei frame liberi
- per caricare un processo che occupa n pagine si devono usare n frame liberi
- traduzione degli indirizzi basata su **page table** (una per processo)
- solo frammentazione interna e solo nell'ultima pagina di ogni processo

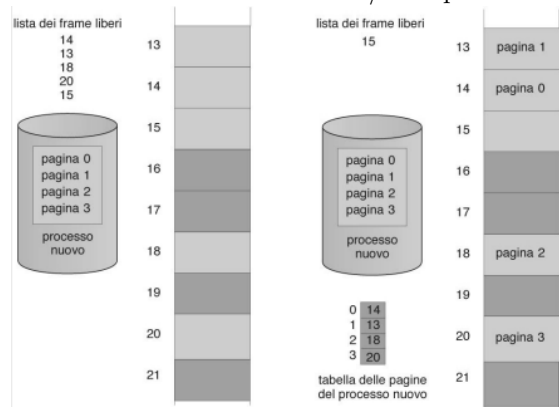
3.2 Architettura

Indirizzo logico viene diviso in due parti:

- **Numero di pagina p** : indice nella page table per determinare numero del frame in memoria fisica
- **Offset (spiazzamento) d** : combinato con numero del frame determina indirizzo fisico



Si tiene traccia di frame liberi / occupati con una **tabella dei frame**:



3.2.1 Gestione dei processi

Gestione processi si complica in quanto:

- **per ogni processo** va gestita una page table: nel PCB si inserisce un puntatore a page table
- context switch diventa più oneroso (vanno garantite informazioni sulla paginazione)
- swapping influenza contenuto della tabella dei frame

3.3 Page Table

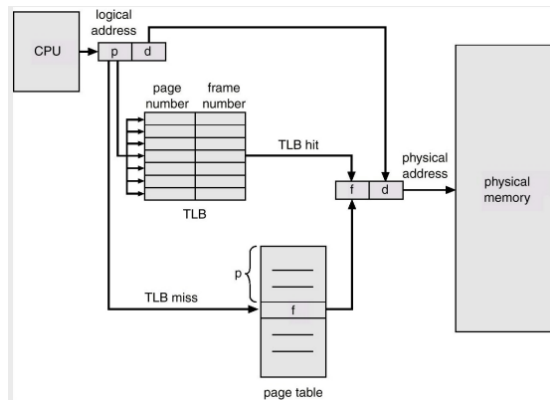
Può essere realizzata in più modi:

- insieme di registri dedicati
 - efficiente ma non adattabile con tabelle grandi

- context switch più complesso
- tabella in memoria
 - localizza tramite **page-table base register (PTBR)**
 - un solo registro da gestire in context switch
 - difetto: per accedere ogni indirizzo servono *due* accessi a memoria
- utilizzo di cache speciale (**translation look-aside buffer (TLB)**), memoria associativa key-value

3.3.1 Page Table con TLB

- TLB contiene solo *parte* della page table
- per tradurre un indirizzo si accede alla TLB
- in caso di **TLB-miss** si dovrà accedere alla vera tabella in memoria
- gestione di quale parte della tabella duplicare nella TLB è fatta con tecniche usate per gestire la cache (FIFO, LRU, ...)
- percentuale di volte che si verifica un **TLB-hit** è detta **hit ratio**
- durante context switch la TLB va svuotata



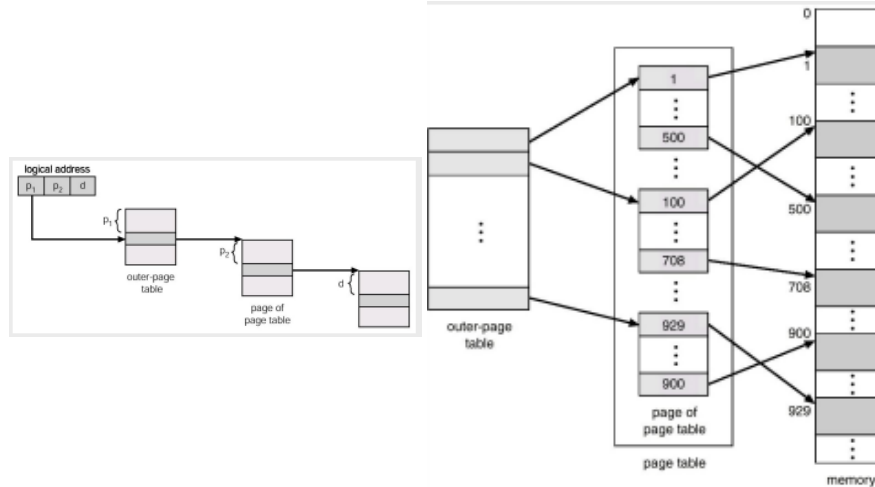
3.3.2 Protezione

Ad ogni elemento della table si associa un **bit di validità** per individuare quali pagine sono nello spazio logico del processo.

Altri bit possono essere nella table per indicare che un particolare frame può essere oggetto di lettura, scrittura,... da parte di un processo.

3.3.3 Struttura

- **Paginazione Gerarchica** → anche tabella delle pagine viene paginata. Esempio di organizzazione **a 2 livelli**: la parte "numero di pagina" p dell'indirizzo logico è a sua volta divisa in due porzioni p_1 e p_2



Sono possibili organizzazioni con **più livelli** (architetture ARM usano paginazione gerarchica fino a 4 livelli)

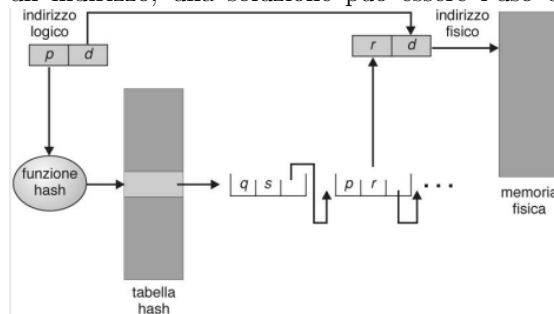
Esempio: considera indirizzi logici di 32 bit con pagine da 4K, l'indirizzo è suddiviso in:

- 20 bit per numero di pagina
- 12 bit di offset all'interno di una pagina (d)

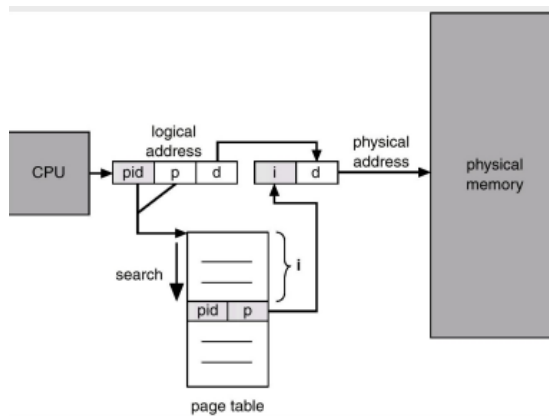
Dato che anche page table è paginata (supponiamo in pagine di 1K), i 20 bit sono divisi in:

- 10 bit per outer-table (p_1)
- 10 bit per offset (p_2) all'interno della pagina della page table

- **Page Table con Hashing** → in caso di aumento di livelli in paginazione gerarchica aumenta numero di accessi a memoria necessari per tradurre un indirizzo, una soluzione può essere l'uso di **organizzazione hash**:



- **Page Table Invertita** → in caso di dimensione della table troppo grande (anche rispetto a numero di frame fisici a disposizione)
 - tabella che memorizza solamente informazioni su pagine realmente utilizzate da processi
 - unica per tutto il sistema
 - un'entry per ogni frame fisico (i), questa contiene indirizzo logico della pagina (p) caricata in quel frame e indicazione del processo possessore (pid)

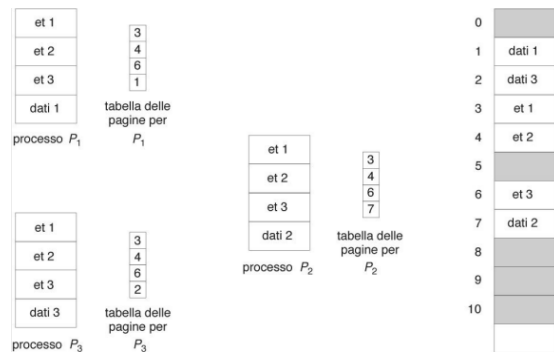


Minor memoria occupata per la table ma **maggior tempo di traduzione**:
 va cercata coppia (pid, p) nella tabella ormai ordinata rispetto a indirizzi fisici (i) → rimedio: implementare inverted page table con hashing.

3.3.4 Pagine Condivise

Tecniche viste permettono di mappare pagine logiche in pagine fisiche ma anche di **mappare diverse pagine logiche nella stessa pagina fisica**:

- si permette condivisione di dati
- si permette condivisione di codice (evitando ripetizioni di codice comune come editor, librerie dinamiche, compilatori...)
- codice condiviso deve essere **rientrante**: non in grado di auto-modificarsi

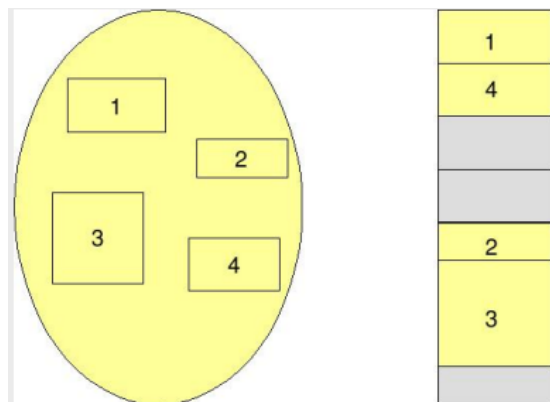


4 Segmentazione

4.1 Generalità

Tecniche finora viste sono accomunate da considerazione della memoria come unico vettore di locazioni

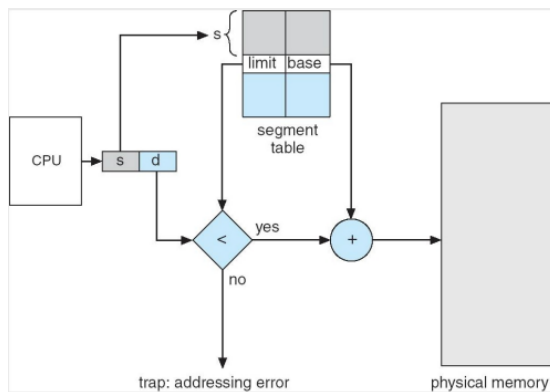
- utente tende a imporre una struttura alla memoria: insieme di **segmenti**, ognuno con **nome**, **dimensione**, e **specifica funzione** (main program, codice procedura, stack, libreria...)
- indirizzi possono essere specificati fornendo nome del segmento e offset al suo interno
- riferimenti interni al segmento sono relativi all'inizio dello stesso
- **compilatore** compila programma dell'utente generando opportuni segmenti, altri possono essere introdotti dal **linker**, il **loader** fornisce "nomi" ai segmenti



Struttura logica del programma e sua allocazione

4.2 Architettura

- Programma visto come insieme di segmenti (non per forza della stessa dimensione)
- ogni segmento allocato in una porzione *contigua* di memoria fisica
- traduzione degli indirizzi avviene tramite **segment table** che, per ogni segmento contiene:
 - **base**: indirizzo fisico dell'inizio del segmento
 - **limite**: lunghezza del segmento
- la segment table è:
 - mantenuta in registri locali
 - o allocata in memoria e gestita tramite **segment-table base register (STBR)** e il **segment-table length register (STLR)**



4.3 Considerazioni

- **rilocalizzazione è dinamica** (a livello di segmenti)
- si può prevedere **condivisione di segmenti** (es. librerie)
- allocazione presenta problema della **frammentazione esterna** (segmenti non hanno tutti stessa dimensione → stessi problemi di allocazione contigua)
- per allocare file si usano first-fit, best-fit...
- **protezione**: per ogni inserimento in segment table si usa un **bit di validità** e eventuali bit per privilegi / modalità d'accesso
- paginazione **trasparente all'utente** (necessità del S.O.) mentre segmentazione è **visibile** (richiesto che utente / compilatore descriva i segmenti)

5 Segmentazione Paginata

5.1 Generalità

Segmentazione e paginazione possono essere combinate per ottenere in un'unica organizzazione i pregi delle due tecniche:

- spazio logico segmentato (riflette struttura logica del programma) come nella segmentazione
- ogni **segmento** è **paginato** (suddiviso in pagine della dimensione del frame fisico)
- per ogni processo si gestisce **una segment table** ed un certo numero di page table: **una per ogni segmento**
- si evita frammentazione esterna a prezzo di una frammentazione (minore) interna (ultima pagina di ogni segmento)

5.2 Architettura

